Relatório do Trabalho 5 de Algoritmos e estrutura de dados II

Métodos de Ordenação

Aluno: ***Felipe Echeverria Vilhalva*** *RGM****:45611***
Professor: ***Fabrício Sérgio de Paula***

1. Introdução

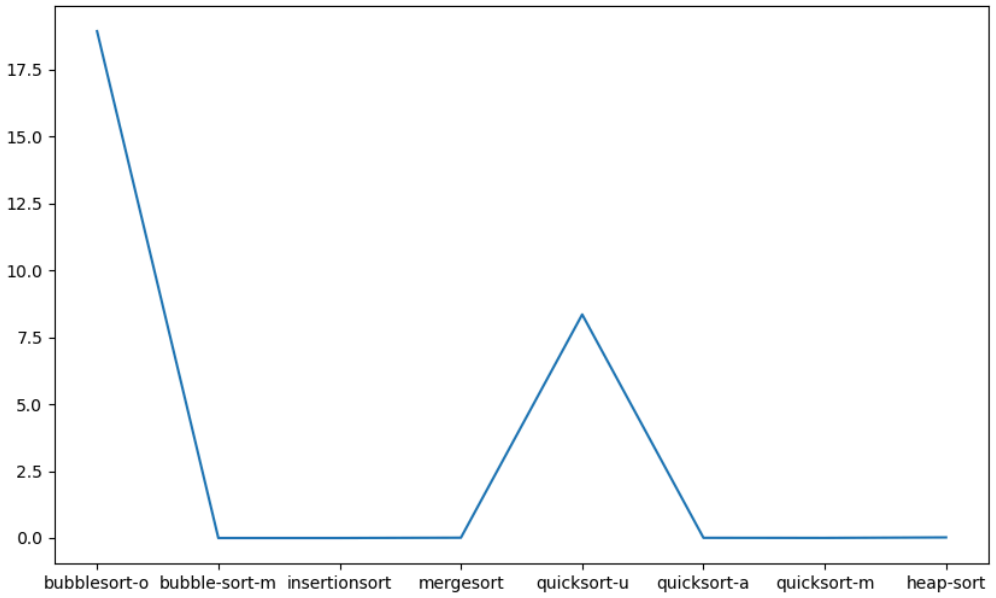
Os métodos de ordenação são algoritmos usados para rearranjar os elementos de uma lista ou vetor em uma ordem específica, geralmente crescente. A escolha do método de ordenação pode impactar significativamente a eficiência do processamento.

Experimentos realizados no **Laboratório 4** usando a máquina **L4M03**, mostraram o desempenho prático de cada um desses algoritmos em diferentes cenários. Esses testes ajudaram a avaliar a eficiência dos métodos em termos de tempo de execução e uso de memória, fornecendo insights valiosos para a escolha do algoritmo mais adequado para diferentes aplicações.

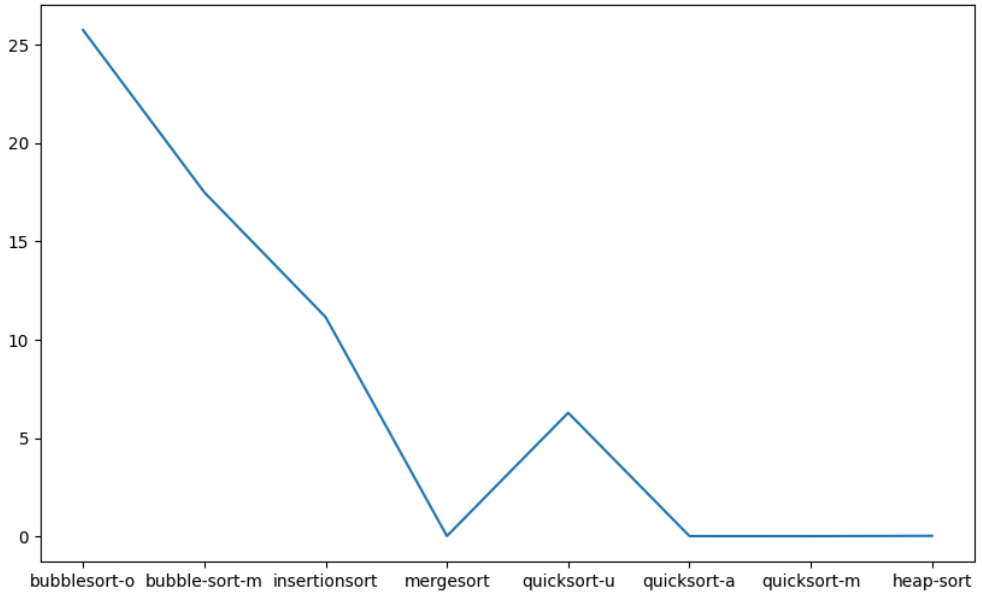
2. Resultados(Gráfico)

Testes com 10, 100, 1000, 10000 e 100000

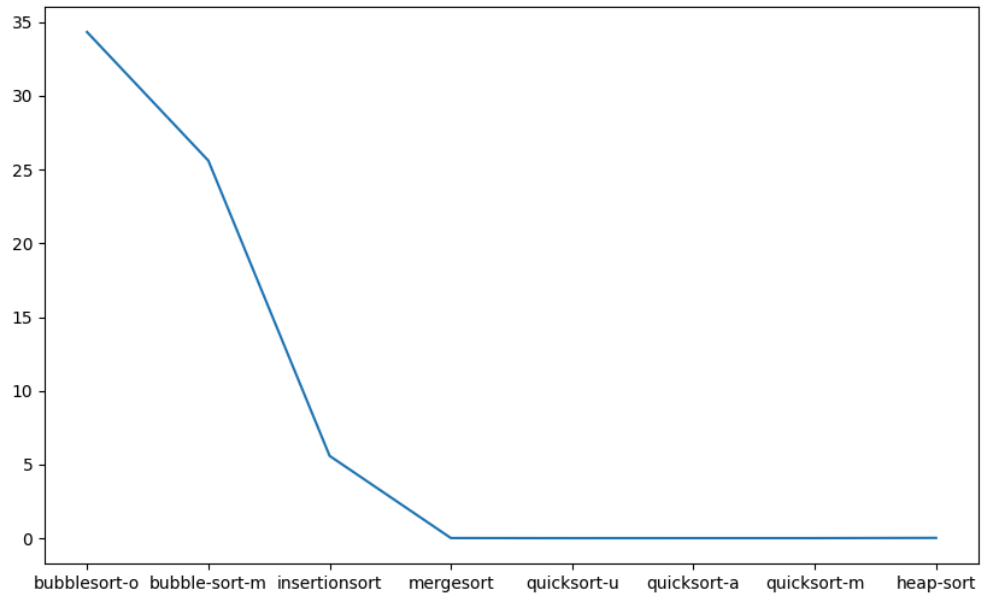
- **crescente**:



- **decrescente:**

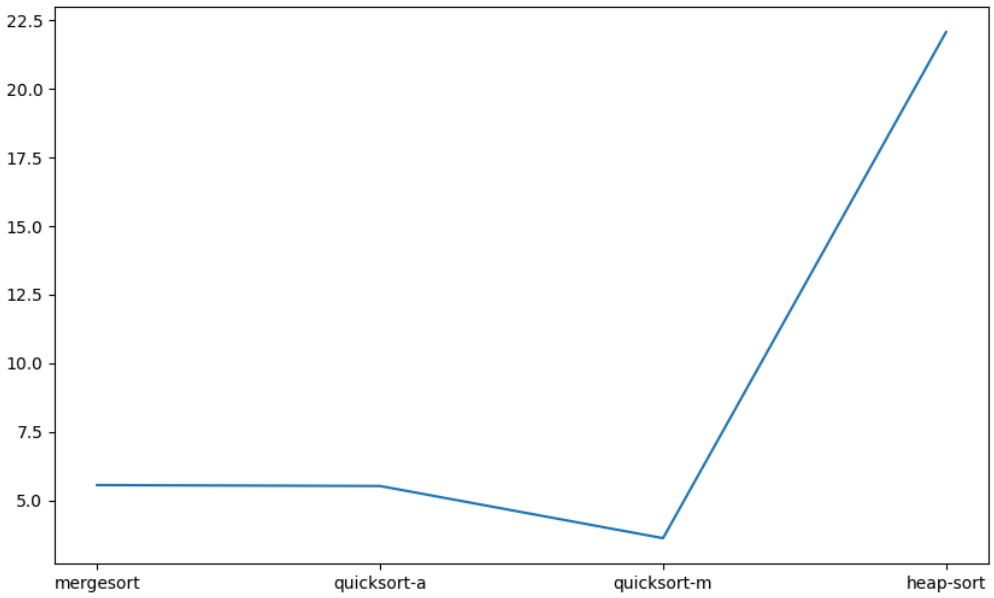


- **aleatório:**

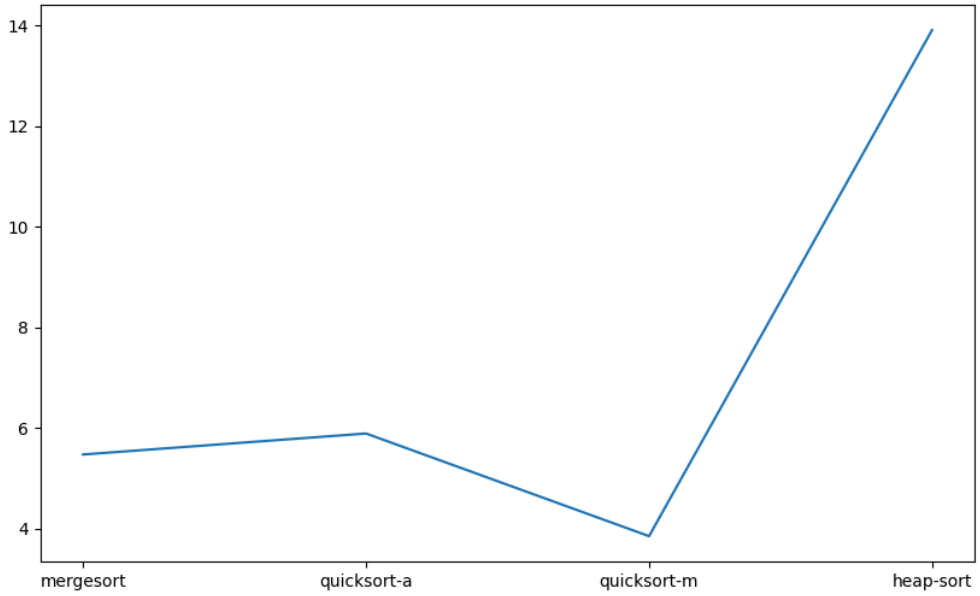


Testes com 500000, 5000000, 50000000:

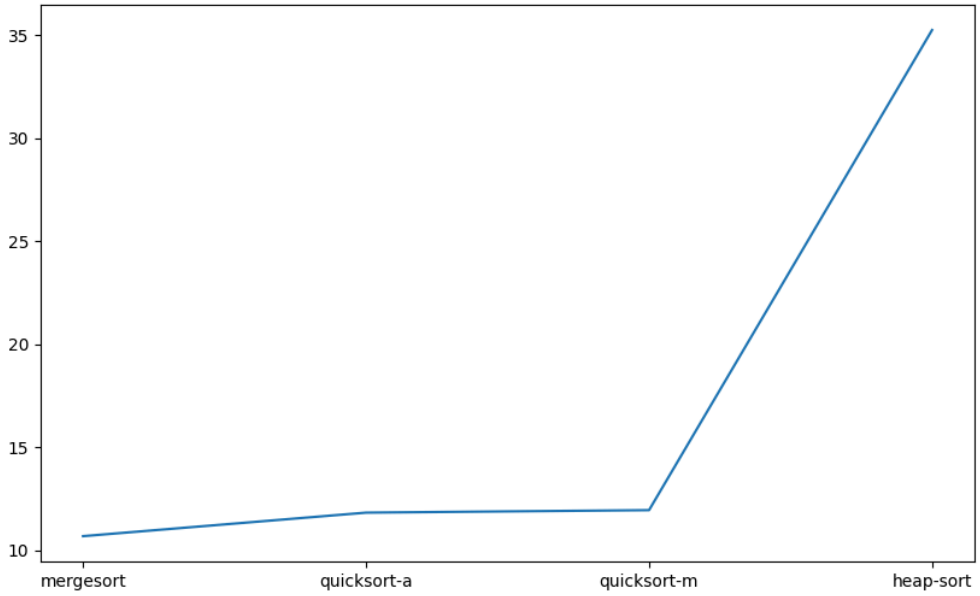
- **crescente**



- **decrescente**



- **aleatória**



3. Discussão(com base nos dados)

3.1. bubble sort original

- Método extremamente ineficiente para grandes quantidades de dados ordenados de maneira crescente, decrescente e aleatória

3.2. bubble sort melhorado

- Método melhorado do bubble sort original
- muito eficiente para grande quantidade de dados ordenados de maneira crescente
- muito ineficiente para grande quantidade de dados ordenados de maneira decrescente e aleatória

3.3. insertion sort

- Método extremamente eficiente para grande quantidade de dados ordenados de maneira crescente
- Método ineficiente para dados ordenados de maneira decrescente e aleatória

3.4. merge sort

- Método muito eficiente para grande quantidade de dados ordenados de maneira crescente e decrescente
- Método extremamente eficiente para dados ordenados de maneira aleatória

3.5. quicksort com pivo como último elemento

- Método ineficiente para grande quantidade de dados ordenados de maneira crescente e decrescente
- Método muito eficiente para grande quantidade de dados ordenados de maneira aleatória

3.6. quicksort com pivo como elemento aleatório

- Método extremamente eficiente para grande quantidade de dados ordenados de maneira crescente, decrescente e aleatória

3.7. quicksort com pivo como mediana de três

- Método extremamente eficiente para grande quantidade de dados ordenados de maneira crescente, decrescente e aleatória

3.8 heap sort

- Método extremamente eficiente para quantidade de dados consideráveis ordenados de maneira crescente, decrescente e aleatória
- Método ineficiente para grande quantidade de dados ordenados de maneira crescente, decrescente e aleatória

4. Dados obtidos(precisos)

4.1. bubble sort original

- 10, 100, 1000, 10000, 100000:
  crescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.002s
  - ➔ 10000: 0.193s
  - ➔ 100000: 18.931s

  decrescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.004s
  - ➔ 10000: 0.262s
  - ➔ 100000: 25.760s

  aleatório:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.003s
  - ➔ 10000: 0.304s
  - ➔ 100000: 34.321s

4.2. bubble sort melhorado

- 10, 100, 1000, 10000, 100000:
  crescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0s
  - ➔ 100000: 0s

  decrescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.003s

- ➔ 10000: 0.177s
- ➔ 100000: 17.503s

aleatório:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0.002s
- ➔ 10000: 0.216s
- ➔ 100000: 25.605s

4.3. insertion sort

- 10, 100, 1000, 10000, 100000:

  crescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0s
  - ➔ 100000: 0s

  decrescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.001s
  - ➔ 10000: 0.114s
  - ➔ 100000: 11.153s

  aleatório:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.001s
  - ➔ 10000: 0.060s
  - ➔ 100000: 5.589s

4.4. merge sort

- 10, 100, 1000, 10000, 100000:

  crescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0.001s
  - ➔ 100000: 0.011s

  decrescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0.001s
  - ➔ 100000: 0.008s

  aleatório:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0.001s
- ➔ 100000: 0.015s

- 500000, 5000000, 50000000:
  crescente:
  - ➔ 500000: 0.045s
  - ➔ 5000000: 0.486s
  - ➔ 50000000: 5.562s

  decrescente:
  - ➔ 500000: 0.045s
  - ➔ 5000000: 0.484s
  - ➔ 50000000: 5.477s

  aleatório:
  - ➔ 500000: 0.049s
  - ➔ 5000000: 0.413s
  - ➔ 50000000: 5.529s

4.5. quicksort com pivo como último elemento

- 10, 100, 1000, 10000, 100000:
  crescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.001s
  - ➔ 10000: 0.085s
  - ➔ 100000: 8.350s

  decrescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0.001s
  - ➔ 10000: 0.064s
  - ➔ 100000: 6.284s

  aleatório:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0.001s
  - ➔ 100000: 0.011s

4.6. quicksort com pivo como elemento aleatório

- 10, 100, 1000, 10000, 100000:
  crescente:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0.001s
- ➔ 100000: 0.007s

decrescente:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0.001s
- ➔ 100000: 0.008s

aleatório:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0s
- ➔ 100000: 0.004s

- 500000, 5000000, 50000000:

crescente:

- ➔ 500000: 0.049s
- ➔ 5000000: 0.413s
- ➔ 50000000: 5.529s

decrescente:

- ➔ 500000: 0.049s
- ➔ 5000000: 0.534s
- ➔ 50000000: 5.894s

aleatório:

- ➔ 500000: 0.104s
- ➔ 5000000: 1.059s
- ➔ 50000000: 11.836s

4.7. quicksort com pivo como mediana de três

- 10, 100, 1000, 10000, 100000:

crescente:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0s
- ➔ 100000: 0.004s

decrescente:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0s
- ➔ 100000: 0.005s

aleatório:

- ➔ 10: 0s
- ➔ 100: 0s
- ➔ 1000: 0s
- ➔ 10000: 0.002s
- ➔ 100000: 0.021s

- 500000, 5000000, 50000000:
  crescente:
  - ➔ 500000: 0.034s
  - ➔ 5000000: 0.337s
  - ➔ 50000000: 3.628s

  decrescente:
  - ➔ 500000: 0.038s
  - ➔ 5000000: 0.344s
  - ➔ 50000000: 3.855s

  aleatório:
  - ➔ 500000: 0.169s
  - ➔ 5000000: 1.081s
  - ➔ 50000000: 11.957s

4.8. heapsort

- 10, 100, 1000, 10000, 100000:
  crescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0.002s
  - ➔ 100000: 0.023s

  decrescente:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0.028s
  - ➔ 100000: 0.021s

  aleatório:
  - ➔ 10: 0s
  - ➔ 100: 0s
  - ➔ 1000: 0s
  - ➔ 10000: 0.002s
  - ➔ 100000: 0.028s

- 500000, 5000000, 50000000:
  crescente:
  - ➔ 500000: 0.169s
  - ➔ 5000000: 1.857s
  - ➔ 50000000: 22.072s

  decrescente:

- ➔ 500000: 0.112s
- ➔ 5000000: 1.592s
- ➔ 50000000: 13.907s

aleatório:

- ➔ 500000: 0.157s
- ➔ 5000000: 2.263s
- ➔ 50000000: 35.245s

5. Conclusão

Desse modo, concluímos que os métodos de ordenação são muito úteis na computação, Logicamente, alguns são mais eficientes que outros, porém leva-se em conta a situação de uso de tais algoritmos.